Lecture 2

Lean: definitions, proofs, and computation

Last week: Dependent types, proofs as terms, and some tactics.

Today: more tactics, induction and recursion, type class instances, computability and decidability.

Inductive definitions

```
inductive Nat where
  | zero : Nat
  | succ (n : Nat) : Nat
```

`Nat.zero` and `Nat.succ (Nat.succ Nat.zero)` are terms.

Every term comes from finitely many applications of these constructors.

Recursion

Our addition recurses on its *second* argument.

```
def plus (m : Nat) : Nat → Nat
  | 0 => m
  | Nat.succ n => Nat.succ (plus m n)

#eval plus 2 3   -- 5
```

$$\begin{aligned}\texttt{plus 2 3} &= \texttt{succ (plus 2 2)} \\ &= \texttt{succ (succ (plus 2 1))} = \cdots = 5.\end{aligned}$$

The recursive call uses the smaller input `n`. Lean checks termination.

Inductive definitions II

```
inductive Even : Nat → Prop where
  | zero : Even 0
  | step (n : Nat) : Even n → Even (n + 2)

example : Even 2 := Even.step 0 Even.zero
```

`Even` is a family of propositions indexed by a natural number.

`Even.zero` proves that zero is even.

`Even.step` turns evidence for n into evidence for $n + 2$.

Proof by induction

Claim: `plus 0 n` $= n$ for every natural number n.

```
theorem zero_plus (n : Nat) : plus 0 n = n := by
  induction n with
  | zero => rfl
  | succ n ih =>
      simp [plus, ih]
```

Base: `plus 0 0` $= 0$ by the definition.

Step: assume `ih : plus 0 n = n`. Then

$$\texttt{plus}\ 0\ (\texttt{succ}\ n) = \texttt{succ}\ (\texttt{plus}\ 0\ n) = \texttt{succ}\ n.$$

`rfl` checks definitional equality. `simp [plus, ih]` unfolds and rewrites.

Proof by contradiction

`¬P` is notation for `P → False`.

```
example (P Q : Prop) (hPQ : P → Q) (hnQ : ¬Q) :
    ¬P := by
  intro hP
  exact hnQ (hPQ hP)
```

Assume P. Obtain Q. Apply $\neg Q$ to obtain `False`.

This argument is constructive: it needs no classical axiom.

A different step: to prove P by assuming $\neg P$ and deriving `False`, we generally use classical reasoning.

Law of excluded middle

```
#check Classical.em   -- (P : Prop) → P ∨ ¬P

example (P : Prop) (hnn : ¬¬P) : P := by
  apply Classical.byContradiction
  intro hnP
  exact hnn hnP
```

Excluded middle says $P \vee \neg P$ for every proposition P.

If P holds, we are done. If $\neg P$ holds, `hnn` gives a contradiction.

`Classical.byContradiction` packages this reasoning.

Type class instances

A **class** specifies data or evidence that a definition needs.
An **instance** supplies it for a particular type. Lean "remembers" instances.

```
class HasAdd (a : Type) where
  add : a -> a

instance : HasAdd Nat where
  add := plus
```

`HasAdd Nat` is a type of records containing a natural number.

The `instance` declaration builds one and registers it for automatic search.

Here the requirement is a chosen starting value. Other classes can require operations or proofs.

Type class instances II

```
theorem add_comm (a : Type) [HasAdd a] [HasComm] (b c : a) : b + c = c + b :=
  sorry

#eval 0 + 7   -- 7
#check (inferInstance : HasAdd Nat)
```

`[HasAdd a]` is an extra argument that Lean supplies by instance search.

At `+`, Lean finds our registered `HasAdd Nat` instance.

`inferInstance` asks Lean to synthesize an instance of the expected type.

Decidability

Lean's definition, with the proposition named `P`:

```
class inductive Decidable (P : Prop) where
  | isFalse (h : ¬P) : Decidable P
  | isTrue  (h : P)  : Decidable P
```

`Decidable P : Type` can guide computation.

A *computable* instance reduces to one of the constructors.

`[Decidable P]` requests an instance that Lean finds automatically.

Natural-number equality and order have computable instances.

Computability

```
def isZero (n : Nat) : Bool := decide (n = 0)

#eval isZero 0       -- true
#eval isZero 7       -- false
#eval if 3 < 5 then 10 else 20  -- 10

example : 3 < 5 := by decide
```

`decide P` computes a Boolean using `[Decidable P]`.

`by decide` proves a goal when its decision reduces to `isTrue`.

A computable definition evaluates by explicit rules to a result.

An `if` with a proposition as its condition also needs decidability.

Decidability is not computability

```
noncomputable def classicalBit (P : Prop) : Bool := by
  letI : Decidable P := Classical.propDecidable P
  exact decide P
```

Classical reasoning supplies a value of `Decidable P`.

It supplies no general algorithm for finding the verdict.

`noncomputable` permits this definition. It does not create executable code.

Classical proofs can still accompany executable programs: proof terms are erased. The obstacle is using noncomputable choices to determine output data.

`letI` supplies an instance locally inside this definition.

Three notions to keep separate

Object	What it provides
$P \vee \neg P$: `Prop`	A logical alternative, available classically for every P.
`Decidable` P : `Type`	A verdict with evidence. Only a computable instance gives an executable decision.
`Bool` : `Type`	A data type with `true` and `false`. Relating a Boolean to a proposition requires a specification or proof.

Today's exercises: human autoformalization

You will do the translation normally asked of an autoformalization system.

Input: an informal mathematical statement and argument.

Your task: choose definitions, state the claim in Lean, and construct a proof that Lean checks.

A small example

"If P implies Q, and Q is false, then P is false."

```
example (P Q : Prop) (hPQ : P → Q) (hnQ : ¬Q) :
    ¬P := by
  -- Translate the argument into a proof.
```

Check that the formal statement says what the original text intended. Lean checks the formal proof; you check the translation. There is a tactic cheat sheet to help!

Attendance

